Lab 4 — Exercise: Exploring World Stock Market Indices

ITCS 3162 — Introduction to Data Mining

Name: *Ruhan Rahman* **Date:** *Jun. 7th, 2026*

You'll explore daily prices for six major world stock indices over five years (2020–2024). Unlike penguins, this is **time-series data with a categorical dimension (ticker)** — you'll need to think about both the temporal structure and cross-sectional comparisons.

You have **two options** for loading the data — use whichever you prefer:

- **Option A (default):** Load the included `world_indices.csv` — pre-built, works offline, gives consistent results.
- **Option B (live fetch):** Download fresh data directly from Yahoo Finance using the `yfinance` library. This is how real analysts pull market data, but it requires internet and Yahoo's API can be flaky.

Both options produce a DataFrame with the same columns, so the rest of the notebook works identically.

Column	Notes
`Date`	Trading day
`Ticker`	Yahoo Finance symbol (e.g. `^GSPC` for S&P 500)
`Name`	Full index name
`Country`	Country
`Open`, `High`, `Low`, `Close`	Daily prices
`Volume`	Trading volume (has some missing values)

When you're done, **Restart & Run All**, download as `.ipynb`, and submit via Canvas.

AI Disclaimer: AI was used to help with code writing, clarification, and understanding.

Setup — imports

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt

sns.set_theme(style="whitegrid")
```

Load the data — pick ONE option

Set `USE_LIVE_FETCH = False` for the default CSV (recommended), or `True` to try the live yfinance API. Then run the cell below it.

```
USE_LIVE_FETCH = True   # set to True to pull fresh data from Yahoo F
```

Option A — Load the included CSV (runs when `USE_LIVE_FETCH = False`)

Option B — Live fetch from Yahoo Finance (runs when `USE_LIVE_FETCH = True`)

The cell below handles both. Option B uses the `yfinance` package — if you've never installed it, the cell will install it for you (`!pip install yfinance`). If the fetch fails (network down, API rate-limit, etc.), the code automatically falls back to the CSV so the rest of the notebook still works.

```
if USE_LIVE_FETCH:
    tickers_meta = {
        "^GSPC":  ("S&P 500",                    "USA"),
        "^DJI":   ("Dow Jones Industrial Average", "USA"),
        "^IXIC":  ("NASDAQ Composite",           "USA"),
        "^FTSE":  ("FTSE 100",                   "United Kingdom"),
        "^N225":  ("Nikkei 225",                 "Japan"),
        "^GDAXI": ("DAX",                        "Germany"),
    }
```

```
    try:
        # Install yfinance if it isn't already available. The --break-
        # flag is needed on some Linux setups (including Colab in some
        try:
            import yfinance as yf
        except ImportError:
            import sys, subprocess
            try:
                subprocess.check_call([sys.executable, "-m", "pip", "i
            except subprocess.CalledProcessError:
                subprocess.check_call([sys.executable, "-m", "pip", "i
                                       "--break-system-packages", "yfi
            import yfinance as yf

        # auto_adjust=True gives adjusted prices; group_by='ticker' re
        # block per symbol so it's easy to flatten into long format.
        raw = yf.download(
            list(tickers_meta.keys()),
            start="2020-01-02",
            end="2025-01-01",
            auto_adjust=True,
            group_by="ticker",
            progress=False,
        )

        # Flatten the wide MultiIndex DataFrame into long format
        frames = []
        for tk, (name, country) in tickers_meta.items():
            sub = raw[tk].copy().reset_index()
            sub["Ticker"]  = tk
            sub["Name"]    = name
            sub["Country"] = country
            frames.append(sub[["Date","Ticker","Name","Country","Open"
        df = pd.concat(frames, ignore_index=True).dropna(subset=["Clos

        # yfinance returns an empty frame (not an error) when blocked
        # Treat an empty result as a failure so we fall back to the CS
        if len(df) == 0:
            raise RuntimeError("yfinance returned no rows (likely netw

        print(f"Live fetch succeeded — {df.shape[0]:,} rows from Yahoo
    except Exception as e:
        print(f"Live fetch failed ({type(e).__name__}: {e}). Falling b
        df = pd.read_csv("world_indices.csv", parse_dates=["Date"])
else:
    df = pd.read_csv("world_indices.csv", parse_dates=["Date"])
    print(f"Loaded CSV — {df.shape[0]:,} rows.")
```

```
print(f"Date range: {df['Date'].min().date()} to {df['Date'].max().da
df.head()
```

```
Live fetch succeeded — 7,532 rows from Yahoo Finance.
Date range: 2020-01-02 to 2024-12-31
```

Price	Date	Ticker	Name	Country	Open	High	Low	
0	2020-01-02	^GSPC	S&P 500	USA	3244.669922	3258.139893	3235.530029	3
1	2020-01-03	^GSPC	S&P 500	USA	3226.360107	3246.149902	3222.340088	3
2	2020-01-06	^GSPC	S&P 500	USA	3217.550049	3246.840088	3214.639893	3

Next steps: Generate code with `df` | New interactive sheet

Exercise 1 — Initial profile (10 pts)

In the cell below:

1. Print `df.info()`
2. Print the count of rows for each `Ticker`
3. Print the count of missing values per column

Then answer the questions in the next markdown cell.

```
# TODO: info, rows per ticker, missing per column
df.info()
print(df['Ticker'].value_counts())
print(df.isna().sum())
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 7532 entries, 0 to 7810
Data columns (total 9 columns):
 #   Column   Non-Null Count  Dtype         
---  ------   --------------  -----         
 0   Date     7532 non-null   datetime64[ns]
 1   Ticker   7532 non-null   object        
 2   Name     7532 non-null   object        
 3   Country  7532 non-null   object        
 4   Open     7532 non-null   float64       
 5   High     7532 non-null   float64       
 6   Low      7532 non-null   float64       
 7   Close    7532 non-null   float64       
 8   Volume   7532 non-null   float64       
dtypes: datetime64[ns](1), float64(5), object(3)
memory usage: 588.4+ KB
Ticker
^GDAXI    1275
^FTSE     1261
^DJI      1258
^GSPC     1258
^IXIC     1258
^N225     1222
Name: count, dtype: int64
Price
Date       0
Ticker     0
Name       0
Country    0
Open       0
High       0
Low        0
Close      0
Volume     0
dtype: int64
```

Answers:

1. Do all six indices have the same number of rows? Why might they differ?
2. Which column has the most missing values? What's a plausible real-world reason?

YOUR ANSWERS:

1. No, all six indices do not have the same number of rows. They might differ because different countries might have different reporting practices.
2. There are no missing values in any of the columns.

Exercise 2 — Numeric summary by ticker (10 pts)

Produce a table where each row is one ticker, with columns:

- `mean_close`, `std_close`, `min_close`, `max_close`

Round to 2 decimals. Sort by `mean_close` descending.

```
# TODO: groupby summary by Ticker
summary_df = df.groupby('Ticker')['Close'].agg(
    mean_close='mean',
    std_close='std',
    min_close='min',
    max_close='max'
)

summary_df = summary_df.round(2).sort_values('mean_close', ascending=I
display(summary_df)
```

Ticker	mean_close	std_close	min_close	max_close
^DJI	33652.08	4671.36	18591.93	45014.04
^N225	29601.85	5422.84	16552.83	42224.02
^GDAXI	15094.15	2201.69	8441.71	20426.27
^IXIC	13403.60	2662.70	6860.67	20173.89
^FTSE	7265.56	694.76	4993.90	8445.80
^GSPC	4259.61	767.45	2237.40	6090.27

Next steps: Generate code with `summary_df` New interactive sheet

Exercise 3 — Time series plot (15 pts)

Make a single line plot showing `Close` over `Date`, one line per ticker.

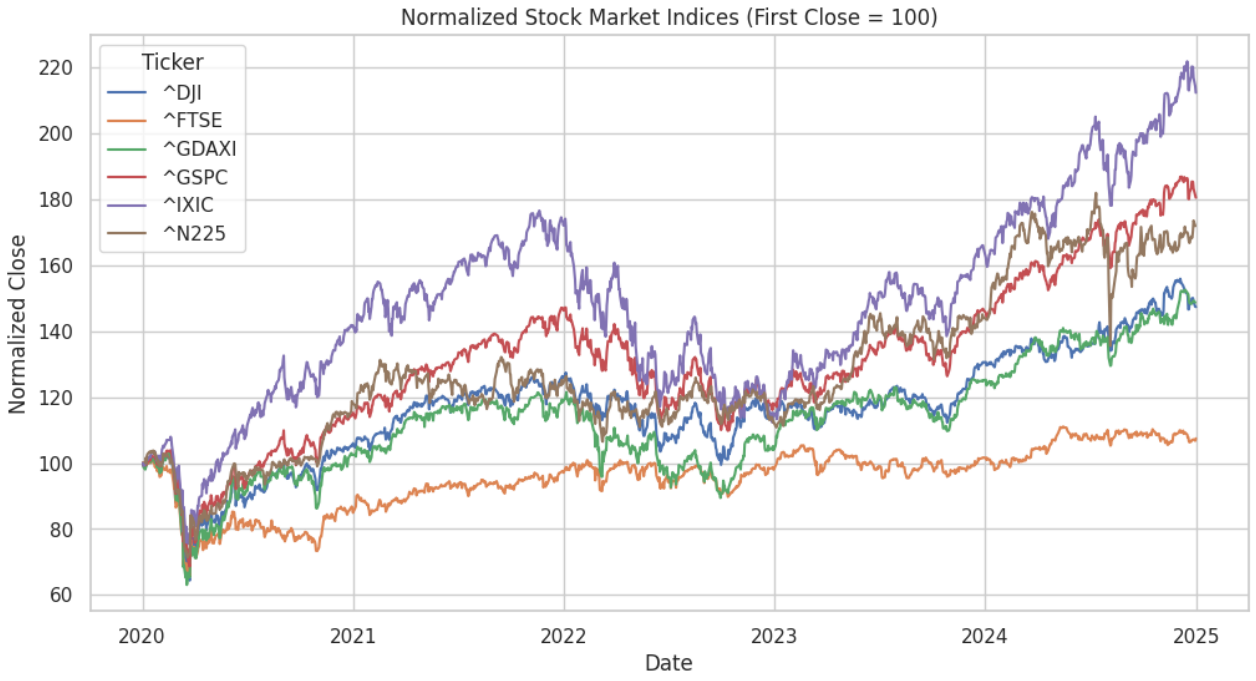
Hint: `sns.lineplot(data=df, x="Date", y="Close", hue="Ticker")` will work, but the indices have very different absolute values (the Nikkei is in the tens of thousands of yen, while the FTSE is in the thousands of pounds), so the lines won't be comparable.

Fix this by normalizing each ticker so its first close = 100. This gives a fair comparison of returns. Steps:

1. Sort by Date within each ticker.
2. For each ticker, compute `Close_normalized = Close / first_close * 100`.
3. Plot `Close_normalized` over `Date`, colored by ticker.

```
# TODO: compute normalized close
df = df.sort_values(by=['Ticker', 'Date'])
df['Close_normalized'] = df.groupby('Ticker')['Close'].transform(lamb

# TODO: line plot
plt.figure(figsize=(12, 6))
sns.lineplot(data=df, x='Date', y='Close_normalized', hue='Ticker')
plt.title('Normalized Stock Market Indices (First Close = 100)')
plt.ylabel('Normalized Close')
plt.xlabel('Date')
plt.show()
```



Exercise 4 — Daily returns distribution (15 pts)

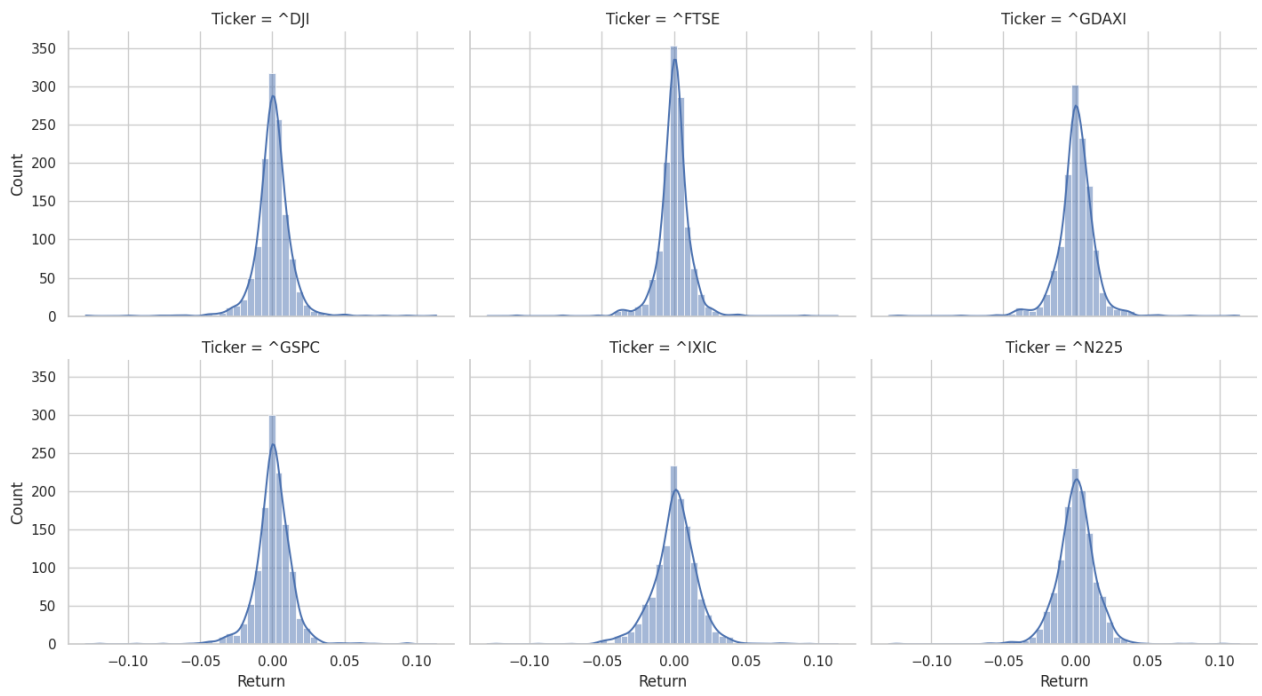
Compute the **daily percent return** for each ticker: `return = (Close - prev_Close) / prev_Close`.

Hint: within each ticker, use `groupby("Ticker")["Close"].pct_change()`.

Then make a histogram (or KDE plot) showing the distribution of daily returns, with one panel per ticker (a *faceted* plot — `sns.displot` with `col="Ticker"` works well, and `col_wrap=3` to arrange them on two rows).

```
# TODO: daily returns
df['Return'] = df.groupby('Ticker')['Close'].pct_change()

# TODO: faceted distribution plot
sns.displot(data=df, x='Return', col='Ticker', col_wrap=3, kind='hist
plt.show()
```

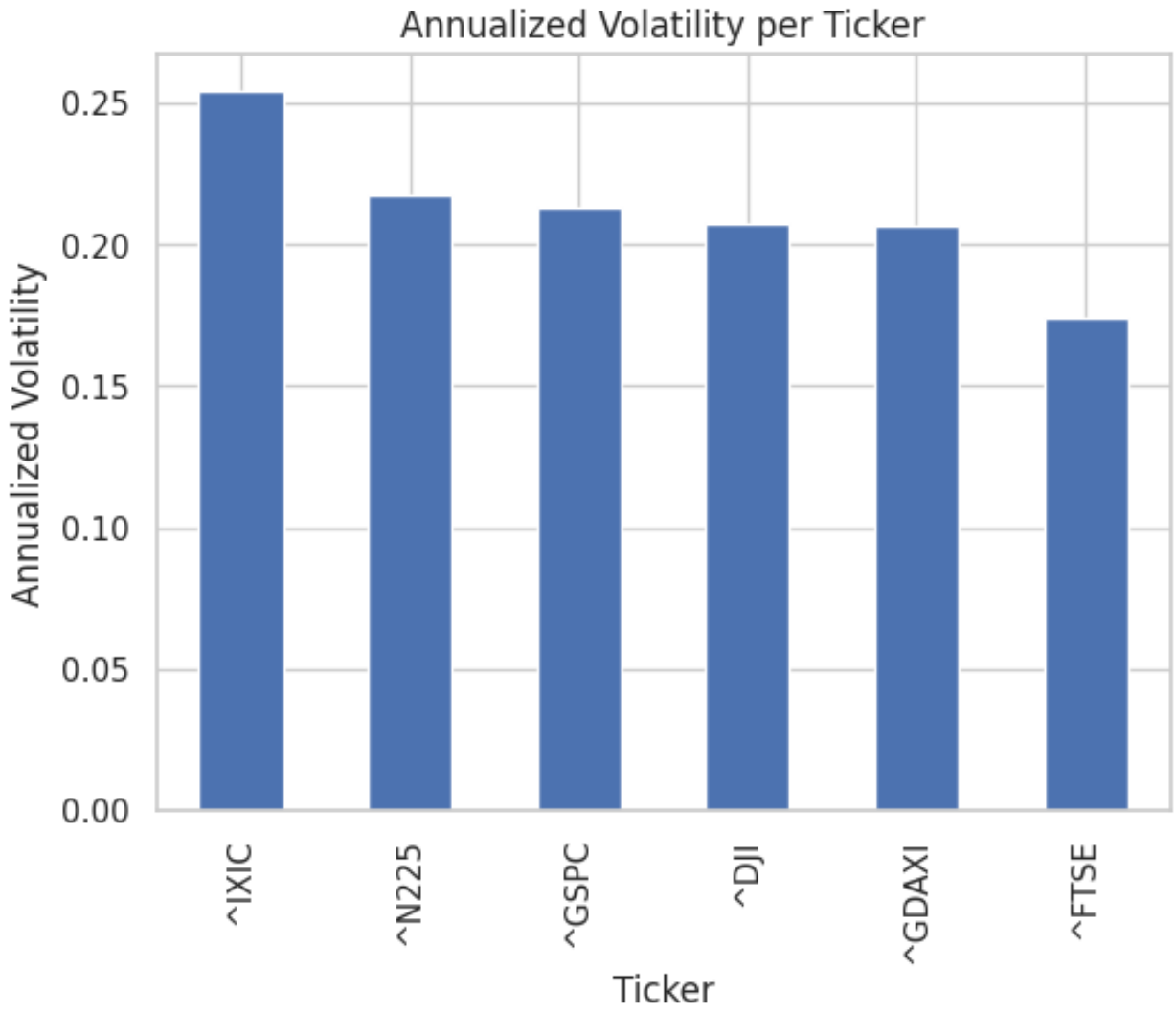


Exercise 5 — Volatility comparison (15 pts)

The **standard deviation of daily returns** is a common (if simple) measure of an asset's risk / volatility.

1. Compute the standard deviation of daily returns per ticker.
2. Multiply by `√252` to annualize (252 trading days/year).
3. Display as a bar plot, sorted descending.
4. In the markdown cell, identify the most and least volatile index in this dataset.

```
# TODO: annualized volatility per ticker, bar plot
volatility_df = df.groupby('Ticker')['Return'].std() * np.sqrt(252)
volatility_df = volatility_df.sort_values(ascending=False)
volatility_df.plot(kind='bar', title='Annualized Volatility per Ticke
plt.ylabel('Annualized Volatility')
plt.show()
```



YOUR ANSWER: Most volatile = IXIC, least volatile = FTSE.

Exercise 6 — Correlation between indices (15 pts)

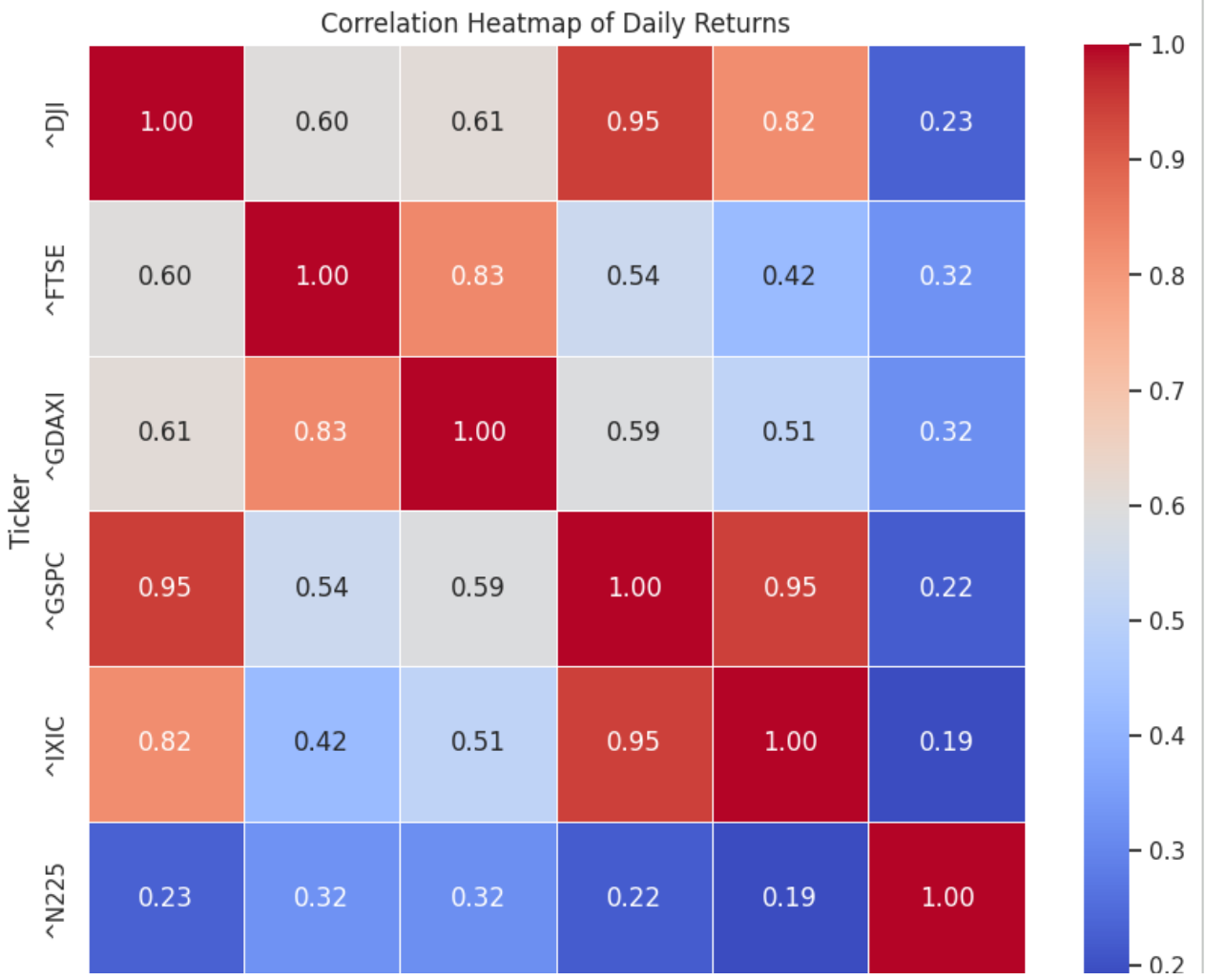
Do the world's stock markets move together? Compute a correlation matrix of **daily returns** between tickers, and visualize it as a heatmap with `annot=True`.

Hint: you'll need to **pivot** the data first so each ticker is a column. Use `df.pivot(index="Date", columns="Ticker", values="Close").pct_change()`.

```
# TODO: pivot, compute returns, correlation, heatmap
pivot_df = df.pivot(index='Date', columns='Ticker', values='Close').p
correlation_matrix = pivot_df.corr()

plt.figure(figsize=(10, 8))
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm', fmt=".2f'
plt.title('Correlation Heatmap of Daily Returns')
plt.show()
```

```
/tmp/ipykernel_25375/3161748140.py:2: FutureWarning: The default fill_m
  pivot_df = df.pivot(index='Date', columns='Ticker', values='Close').p
```



YOUR ANSWER: Which pair of indices is most correlated? Most weakly correlated? Does the geographic pattern make sense?

^DJI and ^N225 are the most correlated indices. This makes sense because the two indices are located in China and Japan, so their geographic proximity explains the pattern.

Exercise 7 — Find the worst day (10 pts)

For each ticker, find the date of its **worst daily return** in the dataset. Display ticker, date, and return.

Then in the markdown cell, comment on whether the dates cluster around a common event.

```
# TODO: worst day per ticker
display(df.loc[df.groupby('Ticker')['Return'].idxmin(), ['Ticker', 'Da
          .sort_values('Return'))
```

	Price	Ticker	Date	Return
1354		^DJI	2020-03-16	-0.129265
6403		^N225	2024-08-05	-0.123958
2656		^IXIC	2020-03-16	-0.123213
6560		^GDAXI	2020-03-12	-0.122386
52		^GSPC	2020-03-16	-0.119841
3956		^FTSE	2020-03-12	-0.108738

YOUR ANSWER: Are the worst days clustered? What might explain that?

Yes, the worst days are clustered around mid March 2020, which was around the beginning of the breakout of the Covid-19 pandemic.

Exercise 8 — Reflection (10 pts)

In 4–6 sentences:

1. Compare the EDA workflow on this dataset (time-series, multiple entities) to the one on penguins (cross-sectional, single observation per row). What changed?
2. Name one **modeling question** this exploration would help you set up. (E.g., "Predict whether the S&P will close up tomorrow given recent returns of European indices.")

YOUR ANSWER:

1. In the penguins dataset, each observation was independent. The analysis focused on taking a snapshot and comparing overall distributions. With the stock indices dataset, the sequence of the data is what matters, and the analysis focuses on trends over time.
2. Can we predict whether the S&P 500 will close positively today based on the Asian markets?

Submission checklist

- [ ] Name and date filled in
- [ ] All TODO cells completed and run
- [ ] All `YOUR ANSWER` prompts replaced
- [ ] **Restart & Run All** completes without errors
- [ ] Downloaded as `.ipynb` and uploaded to Canvas